

强化学习开卷考试算法题复习资料

公式推导 + 原代码函数对照 + 作业题型 + 押题题库

使用建议：考试时先查目录定位算法；算法题按“识别算法 -> 写 target/objective -> 代入计算 -> 写更新值/结论”的顺序答；作业题看第 1 章；DQN/策略梯度/PPO 题看第 5 章和新增第 9 章。

目录

目录	1
1.1 作业 1：爬楼梯问题与矩阵迭代	4
如何写 $x=(I-A)^{-1}b$ 的步骤	5
特征值与收敛性分析	5
1.2 作业 2：TD、SARSA、多步 SARSA、Q-learning	5
2 证明模板与关键词	6
2.1 Bellman 期望方程证明	6
状态价值 Bellman 期望方程	6
动作价值 Bellman 期望方程	6
2.2 策略提升定理证明	7
考试可写步骤	7
2.3 Bellman 最优算子压缩映射证明	7
价值迭代为什么收敛	7
2.4 策略梯度定理证明关键词	8
log-derivative trick	8
2.5 DQN 过高估计与 Double DQN	8
Jensen/最大值偏差证明	8
2.6 PPO 截断目标为什么能限制更新	9
正负优势分别讨论	9
3 全书算法公式地图	9
3.1 基础公式统一表	10

4 动态规划、TD、Dyna-Q 深入	10
4.1 动态规划：代码与公式对照.....	10
4.2 TD 系列：偏差-方差与实验结论.....	11
4.3 Dyna-Q：模型学习和规划.....	12
5 DQN、策略梯度、PPO 深入.....	12
5.1 DQN 原代码对照.....	12
5.2 Double DQN 与 Dueling DQN	12
5.3 REINFORCE 与 Actor-Critic	13
5.4 TRPO 到 PPO	13
5.5 GAE 代码与公式.....	14
6 连续控制与前沿算法	14
6.1 DDPG.....	14
6.2 SAC	14
7 押题题库（含作业题型）	15
题 1 爬楼梯矩阵求解.....	15
题 2 爬楼梯收敛性.....	15
题 3 Bellman 期望方程证明.....	15
题 4 策略提升证明.....	16
题 5 价值迭代一轮.....	16
题 6 SARSA/Q-learning 数值.....	17
题 7 n-step SARSA	17
题 8 Dyna-Q 顺序更新	17
题 9 DQN 目标值.....	18
题 10 Double DQN 目标值.....	18
题 11 Dueling DQN.....	18
题 12 REINFORCE 回报.....	18
题 13 GAE	19

题 14 PPO clip.....	19
题 15 terminated/truncated	19
题 16 Actor-Critic	20
题 17 DDPG 软更新.....	20
题 18 SAC 最大熵	20
8 高频算法对比专题.....	21
8.1 先判断算法属于哪一类.....	21
8.2 高频对比总表.....	21
8.3 对比题答题模板	22
8.4 必背对比：SARSA vs Q-learning	23
8.5 必背对比：DQN、Double DQN、Dueling DQN.....	23
8.6 必背对比：REINFORCE、Actor-Critic、PPO.....	24
8.7 必背对比：DDPG vs SAC	24
8.8 对比题最容易丢分的点.....	25
9 算法题专项押题与答题模板（新增）	25
9.1 通用算法题答题模板	25
9.2 DP：Policy Iteration 与 Value Iteration 押题模板.....	25
9.3 Dyna-Q 顺序更新押题模板.....	26
9.4 MC、TD、SARSA、Q-learning 数值题模板.....	26
9.5 DQN、Double DQN、Dueling DQN 算法题模板.....	27
9.6 REINFORCE 与 Actor-Critic 算法题模板	27
9.7 TRPO 与 PPO 算法题模板	27
9.8 新增算法题押题清单	28
10 考前检索关键词	29
11 常见激活函数总表	29
11.1 怎么选：按网络位置判断	30
11.2 强化学习里怎么用.....	30

11.3 例题：DQN 网络激活函数怎么写.....	31
11.4 例题：PPO 离散策略为什么用 softmax.....	31
11.5 例题：连续动作 Actor 为什么常用 tanh.....	32
11.6 激活函数易错问答.....	32

1 作业题型专题

考试范围与算法题优先级（新增）

本版修订原则：保留旧版结构和作业题；凡是与老师清单算法直接相关或相邻的算法内容都保留，例如 Double/Dueling DQN、n-step 与 DQN/TD 的关系。只删除离本次算法题复习较远的前沿简答和激活函数专题。

老师范围	本版重点补强	算法题常见问法
Model：DP(VI, PI)、Dyna	PI/VI 流程、Bellman 备份、Dyna-Q 真实更新 + planning。	给 P、r 做一轮 VI/PI；按 Dyna 顺序写 Q 更新。
Value：MC、TD、SARSA、Q-learning、DQN	MC 单独补强；TD/SARSA/Q-learning 数值模板；DQN target/loss 与 Double/Dueling。	算 G_t、TD error、Q 更新、DQN target、Dueling 聚合。
Policy：REINFORCE、A-C、TRPO、PPO	策略梯度、baseline、Actor-Critic、TRPO KL 约束、PPO clip。	写梯度/损失；算 ratio；判断 clip；解释 KL/trust region。

1.1 作业 1：爬楼梯问题与矩阵迭代

题型关键词：线性迭代、 $x = Ax + b$ 、不动点、 $(I-A)^{-1}$ 、特征值、谱半径、收敛性、状态转移矩阵。

$$x = (x_5, x_4, x_3, x_2, x_1)^T, \quad x = Ax + b$$

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

若 x_i 表示爬到第 i 级楼梯的方法数，并且每次可爬 1 或 2 级，则边界为 $x_1=1, x_2=2$ ，所以可取 $b=(0,0,0,2,1)^T$ 。

$$x_3 = x_2 + x_1 = 3, \quad x_4 = x_3 + x_2 = 5, \quad x_5 = x_4 + x_3 = 8$$

$$x = (8, 5, 3, 2, 1)^T$$

如何写 $x=(I-A)^{-1}b$ 的步骤

1. 从 $x = Ax + b$ 移项得到 $(I-A)x = b$ 。
2. 若 $I-A$ 可逆，则 $x = (I-A)^{-1}b$ 。
3. 这里 A 是严格上三角矩阵， $A^5=0$ ，所以 A 是幂零矩阵。
4. 因此 $(I-A)^{-1}=I+A+A^2+A^3+A^4$ 。考试中可写“有限 Neumann 级数”。
5. 代入 b 后得到 $x=(8,5,3,2,1)^T$ 。

特征值与收敛性分析

6. A 是上三角矩阵，特征值等于主对角线元素。
7. 主对角线全为 0，所以全部特征值都是 0。
8. 谱半径 $\rho(A)=\max_i |\lambda_i|=0<1$ 。
9. 线性迭代 $x_{k+1}=Ax_k+b$ 收敛到唯一不动点 $x^*=(I-A)^{-1}b$ 。
10. 误差 $e_k=x_k-x^*$ 满足 $e_{k+1}=Ae_k$ ，所以 $e_k=A^k e_0$ ；由于 $A^5=0$ ，最多 5 次误差归零。

当楼梯级数 N 增大时，若矩阵仍是“只从 i 依赖 $i-1$ 和 $i-2$ ”的严格上三角结构，则 A_N 仍然幂零，所有特征值仍为 0，谱半径仍为 0；变化的是矩阵维度和幂零阶数，最多 N 次完成精确收敛。

状态转移矩阵怎么写：如果题目只问作业中的状态转移矩阵 A ，就写上面的计数递推矩阵；如果把它解释成概率 MDP，则从每个非终止台阶选择爬 1 或 2 级，需把相应转移概率写成 $1/2$ 或按策略 π 加权。计数矩阵不是概率矩阵，每行和不必为 1。

1.2 作业 2：TD、SARSA、多步 SARSA、Q-learning

算法	核心目标	更新公式	关键词
TD(0)	$r + \gamma V(s')$	$V(s) \leftarrow V(s) + \alpha[r + \gamma V(s') - V(s)]$	一步 bootstrap, 评估 V
SARSA	$r + \gamma Q(s', a')$	$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)]$	on-policy, 用实际 a'
n-step SARSA	n 步回报 G	$Q(S_t, A_t) \leftarrow Q + \alpha[G_{t:t+n} - Q]$	MC 与 TD 折中
Q-learning	$r + \gamma \max_a Q(s', a)$	$Q(s, a) \leftarrow Q + \alpha[r + \gamma \max_a Q(s', a) - Q]$	off-policy, 学最优贪心策略

SARSA 与 Q-learning 的迭代差别：SARSA 先按行为策略 ϵ -greedy 选出下一步 a' ，再用 $Q(s', a')$ 更新；Q-learning 不管行为实际下一步选什么，目标里直接取 $\max_a Q(s', a)$ 。

实验结论常用说法：在悬崖漫步中，SARSA 学到的是带探索的行为策略，因此会更保守、倾向远离悬崖；Q-learning 学习目标贪心策略，最优路径更贴近悬崖，但训练时 epsilon 探索可能掉崖，平均回报波动更大。多步 SARSA 通过 n 控制偏差-方差折中， n 太大更像 MC，方差增大； n 太小更像 TD，bootstrap 偏差更强。

第 5 章代码对照：SARSA vs Q-learning

```
class Sarsa:
    def update(self, s0, a0, r, s1, a1):
        td_error = r + self.gamma * self.Q_table[s1, a1] - self.Q_table[s0, a0]
        self.Q_table[s0, a0] += self.alpha * td_error

class QLearning:
    def update(self, s0, a0, r, s1):
        td_error = r + self.gamma * self.Q_table[s1].max() - self.Q_table[s0, a0]
        self.Q_table[s0, a0] += self.alpha * td_error
```

代码里最关键的一行对比：SARSA 是 `self.Q\_table[s1, a1]`，Q-learning 是 `self.Q\_table[s1].max()`。考试写区别时抓住这个关键词即可。

2 证明模板与关键词

2.1 Bellman 期望方程证明

$$G_t = R_{t+1} + \gamma G_{t+1}$$
$$V_{\pi}(s) = E_{\pi}[G_t \mid S_t = s]$$

状态价值 Bellman 期望方程

11. 先写状态价值定义： $V_{\pi}(s) = E_{\pi}[G_t \mid S_t = s]$ 。
12. 写出回报递推： $G_t = R_{t+1} + \gamma G_{t+1}$ 。
13. 把回报递推代入定义： $V_{\pi}(s) = E_{\pi}[R_{t+1} + \gamma G_{t+1} \mid S_t = s]$ 。
14. 因为只给定了 $S_t = s$ ，还没有给定动作，所以先按策略 $\pi(a|s)$ 对动作 $A_t = a$ 求全期望。
15. 写成： $V_{\pi}(s) = \sum_a \pi(a|s) E_{\pi}[R_{t+1} + \gamma G_{t+1} \mid S_t = s, A_t = a]$ 。
16. 给定 s, a 后，环境会以 $p(s', r|s, a)$ 产生下一状态 s' 和奖励 r ，所以再对 s', r 求和。
17. 写成： $\sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) E_{\pi}[R_{t+1} + \gamma G_{t+1} \mid s, a, s', r]$ 。
18. 在条件 s, a, s', r 下， $R_{t+1} = r$ 是已知量，可以从期望里拿出来。
19. 得到： $\sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) [r + \gamma E_{\pi}(G_{t+1} \mid S_{t+1} = s')]$ 。
20. 根据状态价值定义， $E_{\pi}(G_{t+1} \mid S_{t+1} = s') = V_{\pi}(s')$ 。
21. 把这一步替换回去，即得到 Bellman 期望方程。
22. 最后单独写结论： $V_{\pi}(s) = \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) [r + \gamma V_{\pi}(s')]$
 $V_{\pi}(s) = \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) [r + \gamma V_{\pi}(s')]$
 $Q_{\pi}(s, a) = \sum_{s', r} p(s', r|s, a) [r + \gamma \sum_{a'} \pi(a'|s') Q_{\pi}(s', a')]$

动作价值 Bellman 期望方程

23. 先写动作价值定义： $Q_{\pi}(s, a) = E_{\pi}[G_t \mid S_t = s, A_t = a]$ 。

24. 代入回报递推: $Q_{\pi}(s,a)=E_{\pi}[R_{t+1}+\gamma G_{t+1}|s,a]$ 。
25. 此时动作 a 已经给定, 所以第一层不用再对 $\pi(a|s)$ 求和。
26. 给定 s,a 后, 由环境转移概率 $p(s',r|s,a)$ 对 s',r 求和。
27. 得到: $\sum_{s',r} p(s',r|s,a)[r+\gamma E_{\pi}(G_{t+1}|S_{t+1}=s')]$ 。
28. 下一时刻如果只写状态价值, 可直接替换成 $V_{\pi}(s')$ 。
29. 如果想完全写成 Q 的形式, 再用 $V_{\pi}(s')=\sum_{a'} \pi(a'|s')Q_{\pi}(s',a')$ 。
30. 代入后得到动作价值 Bellman 期望方程。

2.2 策略提升定理证明

若对所有 $s: Q_{\pi}(s,\pi'(s)) \geq V_{\pi}(s)$, 则 $V_{\pi'}(s) \geq V_{\pi}(s)$

考试可写步骤

31. 先写原策略价值和动作价值的关系: $V_{\pi}(s)=\sum_a \pi(a|s)Q_{\pi}(s,a)$ 。
32. 若 π' 是对 Q_{π} 贪心的策略, 则 $\pi'(s)=\operatorname{argmax}_a Q_{\pi}(s,a)$ 。
33. 最大值不小于加权平均, 所以 $Q_{\pi}(s,\pi'(s))=\max_a Q_{\pi}(s,a) \geq \sum_a \pi(a|s)Q_{\pi}(s,a)=V_{\pi}(s)$ 。
34. 于是得到关键不等式: $V_{\pi}(s) \leq Q_{\pi}(s,\pi'(s))$ 。
35. 展开右边: $Q_{\pi}(s,\pi'(s))=E[R_{t+1}+\gamma V_{\pi}(S_{t+1}) | S_t=s, A_t=\pi'(s)]$ 。
36. 因为 π' 在状态 s 选择 $\pi'(s)$, 所以上式也可以写成 $E_{\pi'}[R_{t+1}+\gamma V_{\pi}(S_{t+1})|S_t=s]$ 。
37. 对下一状态 S_{t+1} 继续使用同样的不等式: $V_{\pi}(S_{t+1}) \leq Q_{\pi}(S_{t+1},\pi'(S_{t+1}))$ 。
38. 代入后得到: $V_{\pi}(s) \leq E_{\pi'}[R_{t+1}+\gamma Q_{\pi}(S_{t+1},\pi'(S_{t+1}))]$ 。
39. 再展开这个 Q : $Q_{\pi}(S_{t+1},\pi'(S_{t+1}))=E[R_{t+2}+\gamma V_{\pi}(S_{t+2})]$ 。
40. 反复代入, 得到 $V_{\pi}(s) \leq E_{\pi'}[R_{t+1}+\gamma R_{t+2}+\gamma^2 R_{t+3}+\dots]$ 。
41. 右边正是按新策略 π' 执行的回报期望, 即 $V_{\pi'}(s)$ 。
42. 因此 $V_{\pi'}(s) \geq V_{\pi}(s)$, 说明贪心提升后的策略不劣于原策略。

2.3 Bellman 最优算子压缩映射证明

$$(T^*V)(s)=\max_a \sum_{s',r} p(s',r|s,a)[r+\gamma V(s')]$$

价值迭代为什么收敛

43. 取任意两个价值函数 U,V , 目标是证明 T^* 会把二者距离缩小 γ 倍。
44. 固定任意状态 s , 写出二者差值: $|(T^*V)(s)-(T^*U)(s)|$ 。
45. 把 T^* 的定义代入: 二者都是对动作 a 取 $\max$ 的 Bellman 备份。
46. 使用最大值不等式: $|\max_a f_a - \max_a g_a| \leq \max_a |f_a - g_a|$ 。
47. 于是只需比较同一个动作 a 下的备份差。
48. 同一个动作下, 奖励 r 不依赖 U,V , 所以奖励项相消。
49. 剩下 $\gamma \sum_{s',r} p(s',r|s,a)[V(s')-U(s')]$ 。
50. 把绝对值放进去并用三角不等式: $\leq \gamma \sum_{s',r} p(s',r|s,a)|V(s')-U(s')|$ 。

51. $|V(s') - U(s')|$ 不超过无穷范数 $\|V - U\|_{\infty}$ 。
52. 概率求和为 1，所以该项 $\leq \gamma \|V - U\|_{\infty}$ 。
53. 对所有 s 取最大，得到 $\|T^*V - T^*U\|_{\infty} \leq \gamma \|V - U\|_{\infty}$ 。
54. 当 $\gamma < 1$ 时， T^* 是压缩映射，所以存在唯一不动点 V^* ，价值迭代 $V_{k+1} = T^*V_k$ 收敛到 V^* 。

2.4 策略梯度定理证明关键词

$$J(\theta) = E_{\{\tau \sim \pi_{\theta}\}}[R(\tau)]$$

$$\text{grad } J = E[\text{grad log } p_{\theta}(\tau) R(\tau)]$$

log-derivative trick

55. 先把目标函数写成轨迹期望： $J(\theta) = E_{\{\tau \sim \pi_{\theta}\}}[R(\tau)]$ 。
 56. 把期望展开成求和： $J(\theta) = \sum_{\tau} p_{\theta}(\tau) R(\tau)$ 。
 57. 对 θ 求梯度： $\text{grad } J(\theta) = \sum_{\tau} \text{grad } p_{\theta}(\tau) R(\tau)$ 。
 58. 在每一项中乘除 $p_{\theta}(\tau)$ ： $\text{grad } p_{\theta}(\tau) = p_{\theta}(\tau) * \text{grad } p_{\theta}(\tau) / p_{\theta}(\tau)$ 。
 59. 利用恒等式 $\text{grad log } p = \text{grad } p / p$ ，得到 $\text{grad } p_{\theta}(\tau) = p_{\theta}(\tau) \text{grad log } p_{\theta}(\tau)$ 。
 60. 代回去： $\text{grad } J = \sum_{\tau} p_{\theta}(\tau) \text{grad log } p_{\theta}(\tau) R(\tau)$ 。
 61. 把求和重新写回期望： $\text{grad } J = E[\text{grad log } p_{\theta}(\tau) R(\tau)]$ 。
 62. 展开轨迹概率： $p_{\theta}(\tau) = p(s_0) \prod_t \pi_{\theta}(a_t | s_t) P(s_{t+1} | s_t, a_t)$ 。
 63. 环境初始分布 $p(s_0)$ 和转移 P 不含 θ ，求 log 后对 θ 的梯度为 0。
 64. 所以 $\text{grad log } p_{\theta}(\tau) = \sum_t \text{grad log } \pi_{\theta}(a_t | s_t)$ 。
 65. 代入后得到 REINFORCE 形式： $\text{grad } J = E[\sum_t \text{grad log } \pi_{\theta}(a_t | s_t) R(\tau)]$ 。
 66. 实际算法常把完整 $R(\tau)$ 换成从 t 开始的 G_t ，或换成 $Q/A/\text{baseline}$ 来降低方差。
- $$\text{grad } J(\theta) = E[\sum_t \text{grad log } \pi_{\theta}(a_t | s_t) G_t]$$

2.5 DQN 过高估计与 Double DQN

Jensen/最大值偏差证明

67. 假设真实动作价值为 Q_a ，网络估计为 $X_a = Q_a + \epsilon_a$ 。
68. 设估计噪声均值为 0，即 $E[\epsilon_a] = 0$ 。
69. 因此每个动作的估计是无偏的： $E[X_a] = Q_a$ 。
70. 但 DQN 目标里不是先求期望，而是先取 max： $\max_a X_a$ 。
71. max 函数是凸函数，所以由 Jensen 不等式有 $E[\max_a X_a] \geq \max_a E[X_a]$ 。
72. 把 $E[X_a] = Q_a$ 代入，得到 $E[\max_a X_a] \geq \max_a Q_a$ 。
73. 这说明即使每个动作估计本身无偏，最大值估计也会偏大。
74. 直观理解：max 更容易选中噪声正向偏大的动作。
75. Vanilla DQN 目标 $y = r + \gamma \max_a Q_{\text{target}}(s', a)$ 同时用同一组估计来选动作和评估动作，所以容易过高估计。
76. Double DQN 拆成两步：在线网络选 $a^* = \text{argmax}_a Q_{\text{online}}(s', a)$ 。
77. 目标网络只评估被选动作： $y = r + \gamma Q_{\text{target}}(s', a^*)$ 。

78. 选动作噪声和评估噪声被拆开，因此过高估计会减轻。

2.6 PPO 截断目标为什么能限制更新

```
r_t(theta)=pi_theta(a_t|s_t)/pi_old(a_t|s_t)
L_clip=E[min(r_t A_t, clip(r_t,1-eps,1+eps)A_t)]
```

正负优势分别讨论

- 79. 先解释 r_t : 它是新策略概率除以旧策略概率, $r_t > 1$ 表示动作概率被提高, $r_t < 1$ 表示动作概率被降低。
- 80. 再解释 A_t : $A_t > 0$ 表示该动作比平均好, $A_t < 0$ 表示该动作比平均差。
- 81. 当 $A_t > 0$ 时, 优化希望增大 r_t , 也就是提高好动作概率。
- 82. 未截断项 $r_t A_t$ 会随 r_t 增大而增大。
- 83. 但当 $r_t > 1 + \text{eps}$ 时, $\text{clip}(r_t) = 1 + \text{eps}$, 截断项固定为 $(1 + \text{eps})A_t$ 。
- 84. 目标取 $\min$, 因此会选择较小的截断项, 继续增大 r_t 不再带来收益。
- 85. 当 $A_t < 0$ 时, 优化希望减小 r_t , 也就是降低坏动作概率。
- 86. 因为 A_t 为负, r_t 越小, $r_t A_t$ 越接近 0, 目标看起来越大。
- 87. 但当 $r_t < 1 - \text{eps}$ 时, $\text{clip}(r_t) = 1 - \text{eps}$ 。
- 88. 由于 $A_t < 0$, $\text{clip}(r_t)A_t$ 比 $r_t A_t$ 更小, $\min$ 会选择更保守的截断项。
- 89. 所以正优势限制“概率涨太多”, 负优势限制“概率降太多”。
- 90. 结论: PPO clip 让新策略不能在一批旧数据上偏离旧策略太远。

3 全书算法公式地图

章	主题	核心公式/关键词	考试常问
1	基本概念	agent, environment, state, action, reward, return	强化学习和监督学习区别; 为什么是序贯决策。
2	多臂老虎机	regret, epsilon-greedy, UCB, Thompson	累积懊悔计算; 探索与利用。
3	MDP	Markov property, MRP, MDP, V/Q, Bellman	写 Bellman 方程; 状态价值与动作价值互推。
4	动态规划	policy evaluation/improvement, value iteration	已知模型下求最优策略; 证明收敛。
5	TD	TD(0), SARSA, n-step SARSA, Q-learning	on-policy/off-policy; 数值更新。
6	Dyna-Q	real update + model + planning	n_planning 次更新顺序。

7	DQN	replay buffer, target network, MSE TD loss	目标网络和经验回放的作用。
8	Double/Dueling	selection/evaluation split; $V+A-\text{mean}(A)$	过高估计证明; Dueling 合成 Q。
9	REINFORCE	$\text{grad log pi} * G$	策略梯度推导; 回报从后往前算。
10	Actor-Critic	TD delta as advantage	Actor 和 Critic 各自更新什么。
11	TRPO	surrogate + KL constraint + CG	为什么要 KL 约束; 和 PPO 关系。
12	PPO	ratio, clip, GAE	clip 数值题; old_log_probs detach。
13	DDPG	deterministic policy gradient, soft update	连续动作; 目标网络软更新。
14	SAC	max entropy, twin Q, alpha	最大熵目标; 双 Q 和温度参数。

3.1 基础公式统一表

$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$
 $V_{\pi}(s) = \sum_a \pi(a|s) Q_{\pi}(s, a)$
 $Q_{\pi}(s, a) = \sum_{s', r} p(s', r|s, a) [r + \gamma V_{\pi}(s')]$
 $V^*(s) = \max_a Q^*(s, a)$
 $Q^*(s, a) = \sum_{s', r} p(s', r|s, a) [r + \gamma \max_{a'} Q^*(s', a')]$
 $A_{\pi}(s, a) = Q_{\pi}(s, a) - V_{\pi}(s)$
 occupancy: $\rho_{\pi}(s, a) = \pi(a|s) \sum_t \gamma^t P(S_t = s | \pi)$

4 动态规划、TD、Dyna-Q 深入

4.1 动态规划：代码与公式对照

第 4 章 PolicyIteration.policy_evaluation 核心片段

```

def policy_evaluation(self):
    while 1:
        new_v = [0] * self.env.ncol * self.env.nrow
        for s in range(self.env.ncol * self.env.nrow):
            qsa_list = []
            for a in range(4):
                qsa = 0
                for p, next_state, r, done in self.env.P[s][a]:
                    qsa += p * (r + self.gamma * self.v[next_state] * (1 - done))
            qsa_list.append(self.pi[s][a] * qsa)

```

```
new_v[s] = sum(qsa_list)
V_{k+1}(s)=sum_a pi(a|s) sum_{s'} P(s'|s,a)[r+gamma V_k(s')]
```

- `self.env.P[s][a]` 就是已知模型；动态规划题若没有 P，不能直接套 DP。
- `(1-done)` 是终止处理：到终止状态只拿即时奖励，不再加未来价值。
- `qsa\_list.append(self.pi[s][a] \* qsa)` 对应对动作按策略概率加权。

第 4 章 Policy Iteration.policy_improvement 核心片段

```
def policy_improvement(self):
    for s in range(self.env.nrow * self.env.ncol):
        qsa_list = []
        for a in range(4):
            qsa = 0
            for p, next_state, r, done in self.env.P[s][a]:
                qsa += p * (r + self.gamma * self.v[next_state] * (1 - done))
            qsa_list.append(qsa)
        maxq = max(qsa_list)
        cntq = qsa_list.count(maxq)
        self.pi[s] = [1 / cntq if q == maxq else 0 for q in qsa_list]
```

- `maxq=max(qsa\_list)` 对应贪心策略提升。
- `cntq` 处理并列最优动作；算法题要写清楚并列动作如何分配概率。
- 策略迭代停止条件是策略不再变化；价值迭代停止条件通常是价值变化小于 theta。

4.2 TD 系列：偏差-方差与实验结论

方法	是否等 episode 结束	是否 bootstrap	偏差/方差	实验直觉
MC	是	否	无 bootstrap 偏差，方差大	回合长时更新慢。
TD(0)	否	是，一步	有偏，方差小	在线更新快。
n-step	部分	第 n 步 bootstrap	n 越大越像 MC	n 是折中旋钮。
SARSA	否	用实际 a'	on-policy	悬崖环境更保守。
Q-learning	否	用 max a	off-policy	学最优贪心但探索时更易掉崖。

第 5 章 n-step SARSA 核心片段

```
class nstep_Sarsa:
    def update(self, s0, a0, r, s1, a1, done):
        self.state_list.append(s0)
        self.action_list.append(a0)
        self.reward_list.append(r)
        if len(self.state_list) == self.n:
```

```

G = self.Q_table[s1, a1]
for i in reversed(range(self.n)):
    G = self.gamma * G + self.reward_list[i]
s = self.state_list.pop(0)
a = self.action_list.pop(0)
self.reward_list.pop(0)
self.Q_table[s, a] += self.alpha * (G - self.Q_table[s, a])

```

代码解释：`G = Q(s\_{t+n}, a\_{t+n})` 是 bootstrap 尾项；反向循环不断乘 gamma 加 reward，得到 n 步回报；最后用 `alpha\*(G-Q)` 更新最早的状态动作。

4.3 Dyna-Q：模型学习和规划

第 6 章 DynaQ.update 核心片段

```

def update(self, s0, a0, r, s1):
    self.q_learning(s0, a0, r, s1)
    self.model[(s0, a0)] = r, s1
    for _ in range(self.n_planning):
        (s, a), (r, s_) = random.choice(list(self.model.items()))
        self.q_learning(s, a, r, s_)

```

- 第一行：真实样本的一次 Q-learning。
- 第二行：把真实样本写入经验模型；实验代码用字典，只保存某个 (s,a) 最近一次的 (r,s')。
- 循环：从模型随机抽旧样本做 planning。n_planning=0 时就是 Q-learning。
- 算法题计算时要按顺序逐次更新，因为每次 planning 使用的是已经更新后的 Q 表。

5 DQN、策略梯度、PPO 深入

5.1 DQN 原代码对照

第 7 章 DQN.update 核心片段

```

q_values = self.q_net(states).gather(1, actions)
max_next_q_values = self.target_q_net(next_states).max(1)[0].view(-1, 1)
q_targets = rewards + self.gamma * max_next_q_values * (1 - dones)
dqn_loss = torch.mean(F.mse_loss(q_values, q_targets))
if self.count % self.target_update == 0:
    self.target_q_net.load_state_dict(self.q_net.state_dict())

```

- `q\_net(states)` 输出每个动作的 Q；`gather(1, actions)` 只取真实执行动作对应的 Q(s,a)。
- `target\_q\_net` 只计算目标 y，不直接被当前梯度更新。
- `load\_state\_dict` 是硬更新目标网络；每 target_update 次复制一次在线网络。

5.2 Double DQN 与 Dueling DQN

第 8 章 Double / Dueling 核心片段

```

if self.dqn_type == 'DoubleDQN':
    max_action = self.q_net(next_states).max(1)[1].view(-1, 1)
    max_next_q_values = self.target_q_net(next_states).gather(1, max_action)
else:

```

```
max_next_q_values = self.target_q_net(next_states).max(1)[0].view(-1, 1)
```

```
Q = V + A - A.mean(1).view(-1, 1)
```

```
Double DQN: y=r+gamma Q_target(s', argmax_a Q_online(s',a))
```

```
Dueling DQN: Q(s,a)=V(s)+A(s,a)-mean_a A(s,a)
```

辨析：Double DQN 改的是目标值构造，Dueling DQN 改的是网络结构；二者可以同时使用。

5.3 REINFORCE 与 Actor-Critic

第 9-10 章代码对照

```
# REINFORCE
G = 0
for i in reversed(range(len(reward_list))):
    log_prob = torch.log(self.policy_net(state).gather(1, action))
    G = self.gamma * G + reward
    loss = -log_prob * G
    loss.backward()
```

```
# Actor-Critic
td_target = rewards + gamma * critic(next_states) * (1 - dones)
td_delta = td_target - critic(states)
actor_loss = mean(-log_probs * td_delta.detach())
critic_loss = MSE(critic(states), td_target.detach())
```

- REINFORCE 必须等整条轨迹结束，从后往前算 G，方差较大。
- Actor-Critic 用 TD 误差近似优势，不必等完整回报；Critic 学 V，Actor 学策略。
- `detach()` 表示 Actor 更新时把 TD 误差当常数，避免梯度穿过 Critic。

5.4 TRPO 到 PPO

TRPO: maximize $E[r_t(\theta) A_t]$, subject to $E[KL(\pi_{old} || \pi_{\theta})] \leq \delta$

- TRPO 用 KL 约束控制策略更新幅度，近似求解要共轭梯度、Hessian-vector product、线性搜索。
- PPO 用 clip 或 KL penalty 近似 TRPO 的信任域思想，实现简单，实验课重点通常考 PPO-截断。

第 12 章 PPO.update 核心片段

```
td_target = rewards + gamma * critic(next_states) * (1 - terminateds)
td_delta = td_target - critic(states)
advantage = compute_advantage(gamma, lmbda, td_delta.cpu()).to(device)
old_log_probs = torch.log(actor(states).gather(1, actions)).detach()
for _ in range(epochs):
    log_probs = torch.log(actor(states).gather(1, actions))
    ratio = torch.exp(log_probs - old_log_probs)
    surr1 = ratio * advantage
    surr2 = torch.clamp(ratio, 1 - eps, 1 + eps) * advantage
    actor_loss = torch.mean(-torch.min(surr1, surr2))
```

- `old\_log\_probs.detach()` 固定旧策略概率，是重要性采样分母。
- `ratio` 是新旧策略概率比；不是奖励比，也不是 Q 值比。

- `terminateds` 不是 `done`：时间截断 truncated 时仍应 bootstrap。

5.5 GAE 代码与公式

rl_utils.compute_advantage

```
def compute_advantage(gamma, lmbda, td_delta):
    td_delta = td_delta.detach().numpy()
    advantage_list = []
    advantage = 0.0
    for delta in td_delta[::-1]:
        advantage = gamma * lmbda * advantage + delta
        advantage_list.append(advantage)
    advantage_list.reverse()
    return torch.tensor(advantage_list, dtype=torch.float)


$$A_t = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$$

```

6 连续控制与前沿算法

6.1 DDPG

第 13 章 DDPG.update 核心片段

```
next_q_values = target_critic(next_states, target_actor(next_states))
q_targets = rewards + gamma * next_q_values * (1 - dones)
critic_loss = mean(MSE(critic(states, actions), q_targets))
actor_loss = -mean(critic(states, actor(states)))
soft_update(actor, target_actor)
soft_update(critic, target_critic)
```

- DDPG 适合连续动作；Actor 输出确定性动作 $\mu(s)$ 。
- Critic 学 $Q(s,a)$ ，Actor 最大化 Critic 对自己动作的评价，所以 loss 写成负号。
- 软更新： $\text{target} \leftarrow (1-\tau)\text{target} + \tau\text{online}$ 。

6.2 SAC

第 14 章离散 SAC.calc_target 思路

```
next_probs = actor(next_states)
entropy = -sum(next_probs * log(next_probs))
min_qvalue = sum(next_probs * min(q1_value, q2_value))
next_value = min_qvalue + alpha * entropy
td_target = rewards + gamma * next_value * (1 - dones)
```

- 最大熵目标：不仅要回报高，还希望策略有足够随机性，便于探索。
- 双 Q 网络取 min，缓解 Q 过高估计。
- α 是温度参数； α 大更重视熵，策略更随机。

7 押题题库（含作业题型）

本章写法：每道题都按“先写已知/定义，再写公式，再逐项代入，最后写结论和易错点”的顺序展开。证明题不要跳步；数值题每一步都保留 target、TD error、更新值，方便阅卷老师给过程分。

题 1 爬楼梯矩阵求解

题目：给定作业 1 中的 A ， $b=(0,0,0,2,1)^T$ ，求 $x=(I-A)^{-1}b$ 。

91. 先说明变量顺序： $x=(x_5, x_4, x_3, x_2, x_1)^T$ ， $b=(0,0,0,2,1)^T$ 。
92. 由 $x=Ax+b$ 的第 5 行得 $x_1=1$ ；第 4 行得 $x_2=2$ 。
93. 第 3 行表示 $x_3=x_2+x_1=2+1=3$ 。
94. 第 2 行表示 $x_4=x_3+x_2=3+2=5$ 。
95. 第 1 行表示 $x_5=x_4+x_3=5+3=8$ 。
96. 所以 $x=(8,5,3,2,1)^T$ 。
97. 如果要求用矩阵逆写：从 $x=Ax+b$ 得 $(I-A)x=b$ 。
98. A 是严格上三角， $A^5=0$ ，所以 $(I-A)^{-1}=I+A+A^2+A^3+A^4$ 。
99. 因此 $x=(I-A)^{-1}b$ ，与递推结果一致。

题 2 爬楼梯收敛性

题目：说明 $x_{k+1}=Ax_k+b$ 是否收敛，并分析谱半径。

100. 先找特征值： A 是上三角矩阵，特征值等于主对角线元素。
101. A 的主对角线全为 0，所以所有特征值 $\lambda_i=0$ 。
102. 谱半径 $\rho(A)=\max_i |\lambda_i|=0$ 。
103. 线性迭代 $x_{k+1}=Ax_k+b$ 的不动点 x^* 满足 $x^*=Ax^*+b$ 。
104. 两式相减： $x_{k+1}-x^*=A(x_k-x^*)$ 。
105. 令误差 $e_k=x_k-x^*$ ，得到 $e_{k+1}=Ae_k$ 。
106. 递推得 $e_k=A^k e_0$ 。
107. 因为 $A^5=0$ ，所以 $e_5=0$ ，即最多 5 次迭代精确到达不动点。
108. 一般结论：若 $\rho(A)<1$ ，则 $A^k \rightarrow 0$ ，迭代收敛；本题是更强的幂零收敛。
109. N 增大时，若仍是只依赖后续两级的严格上三角 A_N ，则全部特征值仍为 0，谱半径仍为 0，但幂零阶数变成 N 。

题 3 Bellman 期望方程证明

题目：从 $V_\pi(s)=E[G_t|S_t=s]$ 推导 Bellman 期望方程。

110. 从状态价值定义开始： $V_\pi(s)=E_\pi[G_t | S_t=s]$ 。
111. 写出回报递推： $G_t=R_{t+1}+\gamma V_\pi(S_{t+1})$ 。

112. 代入定义: $V_{\pi}(s) = E_{\pi}[R_{t+1} + \gamma G_{t+1} \mid S_t = s]$ 。
113. 条件在 $S_t = s$ 时, 动作 A_t 由策略 $\pi(a|s)$ 产生, 所以先对动作求和。
114. 写成: $\sum_a \pi(a|s) E[R_{t+1} + \gamma G_{t+1} \mid S_t = s, A_t = a]$ 。
115. 给定 s, a 后, 环境产生 r 和 s' , 概率为 $p(s', r|s, a)$, 所以再对 s', r 求和。
116. 写成: $\sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) [r + \gamma E(G_{t+1} \mid S_{t+1} = s')]$ 。
117. 根据状态价值定义, $E(G_{t+1} \mid S_{t+1} = s') = V_{\pi}(s')$ 。
118. 代入得到: $V_{\pi}(s) = \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) [r + \gamma V_{\pi}(s')]$ 。
119. 最后说明这就是 Bellman 期望方程, 它把当前状态价值写成一步奖励加下一状态价值的期望。

题 4 策略提升证明

题目: 证明贪心提升策略不劣于原策略。

120. 写原策略的状态价值: $V_{\pi}(s) = \sum_a \pi(a|s) Q_{\pi}(s, a)$ 。
121. 贪心策略 $\pi'(s) = \operatorname{argmax}_a Q_{\pi}(s, a)$, 所以 $Q_{\pi}(s, \pi'(s)) = \max_a Q_{\pi}(s, a)$ 。
122. 最大值不小于加权平均, 因此 $Q_{\pi}(s, \pi'(s)) \geq \sum_a \pi(a|s) Q_{\pi}(s, a) = V_{\pi}(s)$ 。
123. 于是有 $V_{\pi}(s) \leq Q_{\pi}(s, \pi'(s))$ 。
124. 展开右侧 Q : $Q_{\pi}(s, \pi'(s)) = E_{\pi'}[R_{t+1} + \gamma V_{\pi}(S_{t+1}) \mid S_t = s]$ 。
125. 对下一状态同样有 $V_{\pi}(S_{t+1}) \leq Q_{\pi}(S_{t+1}, \pi'(S_{t+1}))$ 。
126. 继续展开, 得到 $V_{\pi}(s) \leq E_{\pi'}[R_{t+1} + \gamma R_{t+2} + \gamma^2 V_{\pi}(S_{t+2})]$ 。
127. 不断重复代入, 有限或折扣无穷情形下余项收敛。
128. 最后得到 $V_{\pi}(s) \leq E_{\pi'}[R_{t+1} + \gamma R_{t+2} + \dots]$ 。
129. 右边就是 $V_{\pi'}(s)$, 所以 $V_{\pi'}(s) \geq V_{\pi}(s)$ 。

题 5 价值迭代一轮

题目: $V_0 = 0$ 。状态 s : a_1 到终止奖 5; a_2 回到 s 奖 1; $\gamma = 0.9$ 。求 V_1, V_2 。

130. 先写价值迭代公式: $V_{k+1}(s) = \max_a [r(s, a) + \gamma V_k(s')]$ 。
131. 第 0 轮给定 $V_0(s) = 0$ 。
132. 动作 a_1 到终止状态, 终止后不 bootstrap, 所以 a_1 目标为 5。
133. 动作 a_2 回到 s , 目标为 $1 + 0.9 \cdot V_0(s) = 1$ 。
134. 因此 $V_1(s) = \max(5, 1) = 5$ 。
135. 第二轮继续用上一轮 V_1 。
136. a_1 目标仍为 5; a_2 目标为 $1 + 0.9 \cdot V_1(s) = 1 + 4.5 = 5.5$ 。
137. 所以 $V_2(s) = \max(5, 5.5) = 5.5$ 。
138. 写答案时强调: 同步价值迭代每一轮用旧 V_k , 不是边算边覆盖。

题 6 SARSA/Q-learning 数值

题目: $\alpha=0.1, \gamma=0.9, Q(s,a)=2, r=1, Q(s',.)=[3,5]$, 实际 a' 的 $Q=3$ 。

- 139. SARSA 是 on-policy, 目标使用实际选出的 a' 。
- 140. 已知实际 a' 的 $Q(s',a')=3$ 。
- 141. SARSA target = $r + \gamma Q(s',a') = 1 + 0.9 \times 3 = 3.7$ 。
- 142. TD error = target - $Q(s,a) = 3.7 - 2 = 1.7$ 。
- 143. 更新后 $Q = 2 + \alpha \times \text{TD error} = 2 + 0.1 \times 1.7 = 2.17$ 。
- 144. Q-learning 是 off-policy, 目标使用 $\max_a Q(s',a)$ 。
- 145. $\max Q(s',.) = \max(3,5)=5$ 。
- 146. Q-learning target = $1 + 0.9 \times 5 = 5.5$ 。
- 147. TD error = $5.5 - 2 = 3.5$ 。
- 148. 更新后 $Q = 2 + 0.1 \times 3.5 = 2.35$ 。
- 149. 最后写区别: SARSA 用实际 a' , Q-learning 用 max。

题 7 n-step SARSA

题目: $\gamma=0.9, n=3$, 奖励 $r_1=1, r_2=2, r_3=3$, 尾项 $Q(S_3, A_3)=10$, 求 G 。

- 150. 写 n-step SARSA 回报: $G_{\{t:t+n\}} = r_1 + \gamma r_2 + \dots + \gamma^{n-1} r_n + \gamma^n Q(S_n, A_n)$ 。
- 151. 本题 $n=3$, 所以 $G = r_1 + \gamma r_2 + \gamma^2 r_3 + \gamma^3 Q(S_3, A_3)$ 。
- 152. 代入 $\gamma=0.9$, 得到 $\gamma^2=0.81, \gamma^3=0.729$ 。
- 153. $G = 1 + 0.9 \times 2 + 0.81 \times 3 + 0.729 \times 10$ 。
- 154. 逐项计算: $1 + 1.8 + 2.43 + 7.29 = 12.52$ 。
- 155. 若题目给旧 Q 和 α , 则继续写 $Q(S_0, A_0) \leftarrow Q + \alpha(G - Q)$ 。
- 156. 若 S_3 是终止状态, 则尾项 $Q(S_3, A_3)$ 应去掉。

题 8 Dyna-Q 顺序更新

题目: $\alpha=0.5, \gamma=0.9, Q(s_0, a_0)=0, Q(s_1, .)=[1,4]$, 真实样本与一次 planning 都为 $(s_0, a_0, 2, s_1)$ 。

- 157. Dyna-Q 的一次 update 包含: 真实 Q-learning 更新、写入模型、 n_{planning} 次模型更新。
- 158. 真实样本目标值 target = $r + \gamma \max_a Q(s_1, a)$ 。
- 159. $\max_a Q(s_1, a)=4$, 所以 target = $2 + 0.9 \times 4 = 5.6$ 。
- 160. 第一次 TD error = $5.6 - Q(s_0, a_0) = 5.6 - 0 = 5.6$ 。
- 161. 真实更新后 $Q(s_0, a_0) = 0 + 0.5 \times 5.6 = 2.8$ 。
- 162. 模型记录 $\text{model}[(s_0, a_0)] = (2, s_1)$ 。
- 163. planning 抽到同一条样本, 目标值仍为 5.6。
- 164. 第二次 TD error = $5.6 - 2.8 = 2.8$ 。
- 165. planning 后 $Q(s_0, a_0) = 2.8 + 0.5 \times 2.8 = 4.2$ 。

166. 结论: Dyna-Q 数值题必须按顺序更新, planning 用的是最新 Q 表。

题 9 DQN 目标值

题目: $r=1, \gamma=0.99, done=0, target\ Q(s')=[11,8]$, 求 Vanilla DQN target。

167. Vanilla DQN 的目标: $y=r+\gamma \max_a Q_target(s',a)*(1-done)$ 。

168. 本题 $done=0$, 所以需要 bootstrap。

169. $target\ Q(s')=[11,8]$, 最大值为 11。

170. 代入: $y=1+0.99*11*(1-0)$ 。

171. 计算: $0.99*11=10.89, y=11.89$ 。

172. 如果 $done=1$, 则 $y=r=1$ 。

题 10 Double DQN 目标值

题目: $online\ Q(s')=[10,12]$, $target\ Q(s')=[11,8]$, $r=1, \gamma=0.99$ 。

173. Double DQN 分两步: 在线网络选动作, 目标网络评估动作。

174. 在线网络 $online\ Q(s')=[10,12]$, 最大的是动作 2。

175. 因此 $a^*=\arg\max_a Q_online(s',a)=\text{动作 2}$ 。

176. 目标网络 $target\ Q(s')=[11,8]$, 动作 2 对应的值是 8。

177. Double DQN target: $y=r+\gamma Q_target(s',a^*)$ 。

178. 代入: $y=1+0.99*8=1+7.92=8.92$ 。

179. 对比 Vanilla DQN 会用目标网络 $\max=11$, 得到 11.89; 这体现 Double DQN 缓解过高估计。

题 11 Dueling DQN

题目: $V=4, A=[1,2,5]$, 求 Q。

180. Dueling DQN 合成公式: $Q(s,a)=V(s)+A(s,a)-\text{mean}_a A(s,a)$ 。

181. 先求优势均值: $\text{mean}(A)=(1+2+5)/3=8/3$ 。

182. 动作 1: $Q_1=4+1-8/3=5-8/3=7/3$ 。

183. 动作 2: $Q_2=4+2-8/3=6-8/3=10/3$ 。

184. 动作 3: $Q_3=4+5-8/3=9-8/3=19/3$ 。

185. 所以 $Q=[7/3, 10/3, 19/3]$, 约为 $[2.333, 3.333, 6.333]$ 。

186. 减去 $\text{mean}(A)$ 的作用是解决 V 和 A 的不可辨识性。

题 12 REINFORCE 回报

题目: 奖励 $[1,2,3]$, $\gamma=0.9$, 求每步 G。

187. REINFORCE 用从当前时刻开始的折扣回报 G_t 。

188. 最后一步没有未来奖励，所以 $G_2=3$ 。
189. 倒数第二步： $G_1=r_2+\gamma G_2=2+0.9*3=4.7$ 。
190. 第一步： $G_0=r_1+\gamma G_1=1+0.9*4.7=5.23$ 。
191. 也可展开写： $G_0=1+0.9*2+0.9^2*3=1+1.8+2.43=5.23$ 。
192. 策略损失通常写 $\text{loss}=-\log \pi(a_t|s_t)G_t$ 。
193. 若有 baseline，则把 G_t 替换成 $G_t-b(s_t)$ 。

题 13 GAE

题目： $\delta=[1,0.5,-0.2]$, $\gamma=0.9$, $\lambda=0.8$ ，求 A。

194. GAE 递推公式： $A_t=\delta_t+\gamma\lambda A_{t+1}$ 。
195. 先算系数 $\gamma\lambda=0.9*0.8=0.72$ 。
196. 从最后一步开始： $A_2=\delta_2=-0.2$ 。
197. 往前一步： $A_1=\delta_1+0.72*A_2=0.5+0.72*(-0.2)$ 。
198. $0.72*(-0.2)=-0.144$ ，所以 $A_1=0.356$ 。
199. 再往前： $A_0=\delta_0+0.72*A_1=1+0.72*0.356$ 。
200. $0.72*0.356=0.25632$ ，所以 $A_0=1.25632$ 。
201. 答案按时间顺序写 $A=[1.25632,0.356,-0.2]$ 。

题 14 PPO clip

题目： $\epsilon=0.2$ 。样本 1 $A=2, r=1.3$ ；样本 2 $A=-2, r=0.7$ 。求 clipped surrogate。

202. PPO 截断目标单样本： $\min(rA, \text{clip}(r, 1-\epsilon, 1+\epsilon)A)$ 。
203. $\epsilon=0.2$ ，所以截断区间是 $[0.8, 1.2]$ 。
204. 样本 1： $A=2>0$ ， $r=1.3$ ，被 clip 成 1.2。
205. 未截断项 $rA=1.3*2=2.6$ 。
206. 截断项 $\text{clip}(r)A=1.2*2=2.4$ 。
207. 取 $\min(2.6, 2.4)=2.4$ 。
208. 样本 2： $A=-2<0$ ， $r=0.7$ ，被 clip 成 0.8。
209. 未截断项 $rA=0.7*(-2)=-1.4$ 。
210. 截断项 $\text{clip}(r)A=0.8*(-2)=-1.6$ 。
211. 取 $\min(-1.4, -1.6)=-1.6$ 。
212. 记忆：正优势限制 r 过大，负优势限制 r 过小。

题 15 terminated/truncated

题目： $r=1, \gamma=0.98, V(s')=10$ 。truncated=True 但 terminated=False，PPO critic target？自然终止呢？

213. PPO critic 目标： $\text{td_target}=r+\gamma V(s')*(1-\text{terminated})$ 。

- 214. 时间截断 $\text{truncated}=\text{True}$ 但 $\text{terminated}=\text{False}$, 表示不是自然终止。
- 215. 此时 $\text{terminated}=0$, 仍然 bootstrap。
- 216. $\text{target}=1+0.98*10*(1-0)=1+9.8=10.8$ 。
- 217. 自然终止时 $\text{terminated}=\text{True}$, 即 $\text{terminated}=1$ 。
- 218. $\text{target}=1+0.98*10*(1-1)=1$ 。
- 219. 实验代码只存 terminated , 不直接用 done , 是为了避免把时间截断误当作终止。

题 16 Actor-Critic

题目: 说明 $\text{actor_loss}=-\log_prob*td_delta.detach$ 的含义。

- 220. Critic 先计算 TD 目标: $r+\gamma V(s')$ 。
- 221. TD 误差 $td_delta=r+\gamma V(s')-V(s)$ 。
- 222. td_delta 可作为优势 $A(s,a)$ 的近似。
- 223. $\text{actor_loss}=-\log_prob*td_delta$ 。
- 224. 若 $td_delta>0$, 说明该动作比预期好; 最小化 loss 会增大 $\log_prob$, 从而提高动作概率。
- 225. 若 $td_delta<0$, 说明该动作比预期差; 最小化 loss 会降低动作概率。
- 226. $.detach()$ 表示把 td_delta 当成常数, 更新 Actor 时不让梯度传入 Critic。
- 227. Critic 自己通过 $\text{MSE}(V(s), td_target)$ 更新。

题 17 DDPG 软更新

题目: $\tau=0.01$, target 参数为 2, online 参数为 5, 软更新后 target?

- 228. DDPG/SAC 常用软更新: $\text{target} <- (1-\tau)*\text{target} + \tau*\text{online}$ 。
- 229. 本题 $\tau=0.01$, 所以 $1-\tau=0.99$ 。
- 230. 旧 $\text{target}=2$, $\text{online}=5$ 。
- 231. $\text{target_new}=0.99*2+0.01*5$ 。
- 232. $0.99*2=1.98$, $0.01*5=0.05$ 。
- 233. $\text{target_new}=2.03$ 。
- 234. 软更新比硬复制更平滑, 能降低目标值震荡。

题 18 SAC 最大熵

题目: 简述 SAC 为什么有 entropy 项以及双 Q 的作用。

- 235. SAC 的目标不是只最大化回报, 还最大化策略熵。
- 236. 策略熵 $H(\pi)$ 越大, 动作分布越随机, 探索越充分。
- 237. 软价值目标可理解为 $Q + \alpha H(\pi)$, 或代码里 $Q - \alpha \log \pi$ 。
- 238. α 是温度参数: α 越大, 越重视熵, 策略越随机。
- 239. SAC 使用两个 Q 网络 critic_1 和 critic_2 。

240. 计算目标时取 $\min(Q1, Q2)$ ，避免像 DQN 那样过高估计。
241. 所以答题关键词是：最大熵探索、温度 α 、双 Q 取 $\min$ 。

8 高频算法对比专题

老师爱问法：对比题不要只写 “一个 on-policy、一个 off-policy”。最好按 “是否需要模型、更新目标、是否 bootstrap、是否等回合结束、策略类型、适用动作空间、优缺点/易错点” 几个维度答。下面的表可以直接背。

8.1 先判断算法属于哪一类

类别	代表算法	是否需要模型	核心学习对象	常考判断
动态规划 DP	策略迭代、价值迭代	需要 P, r	V 或 π	已知环境模型才能直接用；常考 Bellman 备份。
无模型价值法	MC、TD、SARSA、Q-learning	不需要	V 或 Q 表	从采样经验更新；常考 target 和 TD error。
基于模型规划	Dyna-Q	学习一个模型	$Q + \text{model}$	真实更新后再用模型 planning。
深度价值法	DQN、Double、Dueling	不需要	Q 网络	用神经网络近似 Q ；常考 target network、replay。
策略梯度	REINFORCE、Actor-Critic、PPO	不需要	策略 π	直接优化动作概率；常考 log_prob、advantage。
连续控制	DDPG、SAC	不需要	Actor + Critic	动作连续；常考 deterministic/stochastic、双 Q 、熵。

8.2 高频对比总表

对比	共同点	关键区别	考试抓手
DP vs TD	都用 Bellman 思想更新价值	DP 用完整模型求期望；TD 用采样的一步样本	题目给 $P(s' s,a)$ 多半是 DP；只给轨迹样本多半是 TD。
MC vs TD	都可做无模型预测	MC 等 episode 结束，用完整回报；TD 不等结束，用 $r + \gamma V(s')$	MC 无 bootstrap、方差大；TD 有 bootstrap、更新快。

TD(0) vs n-step	都用 bootstrap	TD(0) 只看一步; n-step 看 n 步奖励再接尾项	n 越大越像 MC, n 越小越像 TD。
SARSA vs Q-learning	都更新 $Q(s,a)$	SARSA 用实际 a' ; Q-learning 用 $\max_a Q(s',a)$	SARSA on-policy 更保守; Q-learning off-policy 学最优贪心。
Q-learning vs Dyna-Q	核心真实更新都是 Q-learning	Dyna-Q 额外学习 model, 并做 planning	$n_planning=0$ 时 Dyna-Q 退化为 Q-learning。
Q-learning vs DQN	目标都有 $r+\gamma \max Q$	Q-learning 用 Q 表; DQN 用神经网络近似 Q	DQN 多 replay buffer 和 target network。
DQN vs Double DQN	都用 Q 网络和 TD target	DQN 选动作和估值都用 max; Double 在线选、目标估值	Double 缓解过高估计。
DQN vs Dueling DQN	都输出动作 Q 值	Dueling 把 Q 拆成 $V(s)+A(s,a)-\text{mean}(A)$	Double 改 target; Dueling 改网络结构。
REINFORCE vs Actor-Critic	都属于策略梯度	REINFORCE 用完整回报; AC 用 Critic 的 TD error/优势	REINFORCE 方差大; AC 更新更及时。
TRPO vs PPO	都限制策略更新幅度	TRPO 显式 KL 约束; PPO 用 clip 近似限制	PPO 更简单, 常考 ratio 和 clip。
PPO vs DDPG	都有 Actor-Critic 思想	PPO 多为随机策略、on-policy; DDPG 确定性策略、off-policy	离散动作常 PPO; 连续动作常 DDPG。
DDPG vs SAC	都处理连续控制, 都是 Actor-Critic	DDPG 确定性策略; SAC 随机策略并最大熵	SAC 有 entropy、alpha、双 Q 取 min。
Online RL vs Offline RL	都想学高回报策略	Online 可继续探索; Offline 只能用固定数据集	Offline 难点是分布外动作和外推误差。
BC vs GAIL	都利用专家数据	BC 是监督学习拟合动作; GAIL 用判别器学习模仿奖励	BC 简单但分布偏移; GAIL 匹配占用度量。

8.3 对比题答题模板

242. 第一句先定类: 二者都属于什么类型。例如: SARSA 和 Q-learning 都是无模型 TD 控制算法。
243. 第二句写共同公式骨架: 都用 $Q(s,a) \leftarrow Q(s,a) + \alpha[\text{target} - Q(s,a)]$ 。

244. 第三句写 target 的关键差别。SARSA $\text{target} = r + \gamma Q(s', a')$; Q-learning $\text{target} = r + \gamma \max_a Q(s', a)$ 。
245. 第四句解释策略属性：SARSA 学正在执行的含探索策略，所以是 on-policy; Q-learning 学目标贪心策略，所以是 off-policy。
246. 第五句写实验现象或优缺点：悬崖漫步中 SARSA 更保守，Q-learning 更贴近最优路线但探索时更容易掉崖。
247. 最后补一句易错点：不要把 Q-learning 的 max 写成实际 a' ，也不要写 SARSA 写成 max。

8.4 必背对比：SARSA vs Q-learning

维度	SARSA	Q-learning
更新目标	$r + \gamma Q(s', a')$	$r + \gamma \max_a Q(s', a)$
下一动作	用实际选出的 a'	不管实际动作，直接取最大 Q
策略类型	on-policy, 同策略	off-policy, 异策略
学到的策略	带探索的当前行为策略	最优贪心目标策略
悬崖漫步表现	更保守，远离悬崖	路径更短，但探索时更易掉崖
代码关键词	<code>self.Q_table[s1, a1]</code>	<code>self.Q_table[s1].max()</code>

```
SARSA:      Q <- Q + alpha [r + gamma Q(s', a') - Q]
Q-learning: Q <- Q + alpha [r + gamma max_a Q(s', a) - Q]
```

8.5 必背对比：DQN、Double DQN、Dueling DQN

算法	解决什么问题	核心做法	一句话区别
DQN	Q 表无法处理大状态空间	用神经网络近似 $Q(s, a)$ ，加 replay 和 target network	把 Q-learning 深度化。
Double DQN	DQN 的 max 容易过高估计	online 网络选动作，target 网络评估动作	改的是 target 计算方式。
Dueling DQN	有些状态下动作差异不明显，直接学 Q 不高效	网络分成 V 分支和 A 分支，再合成 Q	改的是网络结构。

248. DQN target: $y = r + \gamma \max_a Q_{\text{target}}(s', a)$ 。
249. Double DQN target: $y = r + \gamma Q_{\text{target}}(s', \text{argmax}_a Q_{\text{online}}(s', a))$ 。

250. Dueling 合成: $Q(s,a)=V(s)+A(s,a)-\text{mean}_a A(s,a)$ 。
251. 考试如果问 “Double 和 Dueling 能否同时用” , 答案是可以, 因为一个改目标, 一个改结构。

8.6 必背对比: REINFORCE、Actor-Critic、PPO

算法	Actor 怎么学	Critic/优势	优缺点
REINFORCE	$\text{grad log pi}(a s) * G_t$	没有 Critic, 可加 baseline	简单但必须等回合结束, 方差大。
Actor-Critic	$-\text{log pi}(a s) * \text{td_delta}$	Critic 学 V , td_delta 近似优势	不必等完整回合, 方差较小但有偏。
PPO	用 $\text{ratio} * \text{advantage}$, 并 clip	常用 GAE 估计优势	限制策略更新幅度, 稳定且实现简单。

252. REINFORCE 看完整回报 G_t , 所以更像 Monte Carlo。
253. Actor-Critic 用 TD error: $\text{delta}=r+\gamma V(s')-V(s)$, 所以结合了 TD 和策略梯度。
254. PPO 在 Actor-Critic 基础上加入 old policy、ratio、clip, 防止策略一步变太多。
255. 写 PPO 对比时一定要提: old_log_probs 要 detach, ratio 是 $\text{pi_new}/\text{pi_old}$ 。

8.7 必背对比: DDPG vs SAC

维度	DDPG	SAC
策略	确定性策略 $a=\mu(s)$	随机策略 $\text{pi}(a s)$
探索	常外加噪声, 如 OU/Gaussian	策略本身随机, 且最大熵鼓励探索
Critic	通常一个 Q	双 Q , 目标取 min
目标	最大化 Q	最大化 $Q + \alpha * \text{entropy}$
稳定性	较敏感, 容易受超参数影响	通常更稳定, 探索更充分
关键词	deterministic policy gradient, soft update	max entropy, alpha, twin Q

8.8 对比题最容易丢分的点

- 不要只背 on-policy/off-policy，要写出 target 公式差别。
- 不要把 Double DQN 和 Dueling DQN 混在一起：Double 改目标，Dueling 改结构。
- 不要说 DQN 输出动作概率；DQN 输出 Q 值，选动作才 argmax。
- 不要把 PPO 的 ratio 说成奖励比；它是新旧策略动作概率之比。
- 不要把 terminated 和 truncated 混用；自然终止不 bootstrap，时间截断通常仍 bootstrap。
- 不要说 SAC 只是 DDPG 加噪声；SAC 是最大熵随机策略，并用双 Q 取 min。

9 算法题专项押题与答题模板（新增）

算法题总写法：先判断算法类别，再写 target/objective，再代入数字算 TD error、ratio、advantage 或 loss，最后写更新后数值和一句算法性质。过程分通常比最终答案更重要。

9.1 通用算法题答题模板

256. 第一行先写：本题属于哪类算法，以及为什么。例如“给出完整转移概率，所以用 DP/VI”；“target 用实际 a' ，所以是 SARSA”。
257. 第二步写核心公式：MC 写 G_t ；TD 写 $r + \gamma V(s')$ ；SARSA 写 $r + \gamma Q(s', a')$ ；Q-learning/DQN 写 $\max \text{target}$ ；PPO 写 ratio 和 clip。
258. 第三步逐项代入：把 α 、 γ 、 r 、 V/Q 、done、advantage、ratio 全部写出来。
259. 第四步写更新值： $\text{new} = \text{old} + \alpha * \text{error}$ ，或 loss/objective 的最终数值。
260. 最后补一句性质：on/off-policy、是否 bootstrap、是否需要模型、是否等 episode 结束、为什么稳定。

9.2 DP: Policy Iteration 与 Value Iteration 押题模板

题型	必须写出的步骤	容易丢分点
一轮 Policy Evaluation	固定 π ，对每个 s 写 $V_{k+1}(s) = \sum_a \pi(a s) \sum p[r + \gamma V_k(s')]$ 。	不要把 max 写进策略评估；评估当前策略不是最优策略。
一轮 Policy Improvement	先用 V_{π} 算每个动作 $q_{\pi}(s, a)$ ，再选 argmax 得新策略。	要逐状态比较所有动作，不要只看即时奖励。
一轮 Value Iteration	对每个状态直接算 $\max_a \sum p[r + \gamma V(s')]$ 。	题目没说异步时，用旧

		V 同步更新。
PI vs VI 对比	PI=评估+提升；VI=最优备份近似合并评估与提升。	不要说二者都不需要模型；DP 必须已知模型。

VI: $V_{k+1}(s) = \max_a \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma V_k(s')]$

PI improvement: $\pi'(s) = \arg\max_a \sum_{s'} P(s'|s,a) [R + \gamma V_{\pi}(s')]$

9.3 Dyna-Q 顺序更新押题模板

261. 真实交互: (s,a,r,s') 到来后, 先按 Q-learning 更新一次 Q(s,a)。
262. 模型学习: $\text{model}[(s,a)] = (r,s')$; 若同一个 (s,a) 再出现, 用最新经验覆盖。
263. 规划更新: 重复 n_{planning} 次, 从见过的 (s,a) 中抽样, 查模型得 r,s', 再做 Q-learning 更新。
264. 若题目给出 planning 抽样顺序, 必须严格按顺序算; 每一步写 target、TD error、新 Q。

9.4 MC、TD、SARSA、Q-learning 数值题模板

算法	target	更新式	识别关键词
MC	G_t	$V \text{ 或 } Q \leftarrow \text{old} + \alpha(G_t - \text{old})$	完整 episode、first/every visit、return。
TD(0)	$r + \gamma V(s')$	$V(s) \leftarrow V(s) + \alpha[\text{target} - V(s)]$	一步 bootstrap、预测 V。
SARSA	$r + \gamma Q(s',a')$	$Q(s,a) \leftarrow Q(s,a) + \alpha[\text{target} - Q(s,a)]$	实际下一动作 a'、on-policy。
Q-learning	$r + \gamma \max_a Q(s',a)$	$Q(s,a) \leftarrow Q(s,a) + \alpha[\text{target} - Q(s,a)]$	max、off-policy、贪心目标。

同一道轨迹比较题: 如果题目让同时算 SARSA 和 Q-learning, 先找实际 a'。SARSA 用这个 a' 的 Q 值; Q-

learning 无视实际 a' ，直接取下一状态所有动作最大 Q 。

9.5 DQN、Double DQN、Dueling DQN 算法题模板

题型	核心公式	答题提醒
DQN target	$y = r + \gamma(1 - \text{done}) \max_a Q_{\text{target}}(s', a)$	done=True 时不 bootstrap, $y=r$ 。
DQN loss	$L = (y - Q_{\text{online}}(s, a))^2$	Q_{online} 负责当前估计, target network 负责稳定目标。
Double DQN target	$a^* = \arg\max_a Q_{\text{online}}(s', a),$ $y = r + \gamma Q_{\text{target}}(s', a^*)$	在线网络选动作, 目标网络估值。
Dueling 聚合	$Q(s, a) = V(s) + A(s, a) - \text{mean}_a A(s, a)$	减均值是为了解决 V/A 不可辨识。

9.6 REINFORCE 与 Actor-Critic 算法题模板

- REINFORCE: 先从后往前算 G_t , 再写 $\theta \leftarrow \theta + \alpha G_t \text{grad} \log \pi(a_t | s_t)$ 。若有 baseline, 改成 $G_t - b(s_t)$ 。
- Actor-Critic: 先算 $\delta = r + \gamma V(s') - V(s)$, Actor 用 δ 更新 $\log \pi$, Critic 用 δ 更新 V 。
- 优势函数题: $A(s, a) = Q(s, a) - V(s)$, 或用 TD error/GAE 近似优势; $A > 0$ 提高动作概率, $A < 0$ 降低动作概率。
- 对比句: REINFORCE 是 MC policy gradient, 方差大但形式简单; A-C 引入 Critic, 方差更小但有 bootstrap 偏差。

REINFORCE: $\text{grad } J = E[\text{grad} \log \pi_{\theta}(a_t | s_t) G_t]$

A-C: $\delta_t = r_t + \gamma V_w(s_{t+1}) - V_w(s_t)$

9.7 TRPO 与 PPO 算法题模板

算法	目标/约束	算法题写法
TRPO	$\max E[r_t(\theta) A_t], \text{ s.t.}$ $E[\text{KL}(\pi_{\text{old}} \pi_{\text{new}})] \leq \delta$	先算 ratio, 再写 KL 约束; 说明 trust region 限制策略大步变化。

PPO-Clip	$E[\min(rA, \text{clip}(r, 1-\epsilon, 1+\epsilon)A)]$	分别讨论 $A>0$ 和 $A<0$; 计算 unclipped 与 clipped 后取 min。
PPO 与 TRPO	PPO 用 clip 近似 TRPO 的 trust region 思想	TRPO 理论更强但复杂; PPO 简单稳定, 常用于算法题。

PPO 正负优势固定话术: $A>0$ 时, ratio 变大代表提高好动作概率, 但超过 $1+\epsilon$ 的收益被截断; $A<0$ 时, ratio 变小代表降低坏动作概率, 但低于 $1-\epsilon$ 后不再继续奖励变小。

9.8 新增算法题押题清单

押题	题目可能给什么	你要输出什么
PI/VI 计算	小 MDP、gamma、初始 V、当前策略	一轮评估/提升或一轮最优备份后的 V 和 pi。
Dyna-Q	真实 transition + planning 抽样序列	每一步 target、TD error、Q 新值、模型表。
MC vs TD	同一条 episode 或一步 transition	G_t 、TD target, 并说明是否 bootstrap。
SARSA vs Q-learning	Q 表、 s, a, r, s' 和实际 a'	两个算法各自 target 和更新值。
DQN target/loss	batch 中的 r、done、 Q_{online} 、 Q_{target}	y、loss, 并说明 replay/target network。
Dueling DQN	$V(s)$ 、各动作 $A(s,a)$	按 $Q=V+A-\text{mean}(A)$ 算每个动作 Q。
REINFORCE	一条轨迹、每步 reward、log_prob	G_t 、策略梯度项或 loss。
Actor-Critic	$V(s)$ 、 $V(s')$ 、r、log_prob	delta、actor loss、critic loss。

TRPO	ratio、advantage、KL、delta	surrogate 值，并判断是否满足 KL 约束。
PPO	ratio、advantage、epsilon	unclipped、clipped、min 结果，分 A 正负解释。

10 考前检索关键词

- 算法题总入口：先判断是否给模型；给 P,r 多半是 DP/VI/PI；只有轨迹样本多半是 MC/TD/SARSA/Q-learning。
- DQN 相关题：DQN target/loss、Double DQN 选动作和估值分离、Dueling DQN 的 V/A 聚合都保留复习。
- Policy 相关题：REINFORCE 看 G_t ；A-C 看 TD error/advantage；TRPO 看 KL 约束；PPO 看 ratio 和 clip。
- $x=Ax+b$, 谱半径：翻作业 1/证明模板；写 $\rho(A)<1$ 收敛， $e_k=A^k e_0$ ，幂零矩阵。
- 已知 $P(s'|s,a)$ ：翻动态规划；写策略评估/提升、价值迭代、Bellman 方程。
- 实际 a' vs max：翻 TD 系列；SARSA 是 on-policy，Q-learning 是 off-policy。
- $n_planning$ ：翻 Dyna-Q；真实更新 + 模型字典 + 多次 planning。
- target network/replay：翻 DQN；MSE TD target，目标网络稳定目标，经验回放去相关。
- overestimation：翻 Double DQN；Jensen 不等式，在线网络选动作、目标网络评估。
- V 和 A 两支：翻 Dueling DQN； $Q=V+A-\text{mean}(A)$ 。
- ratio/clip：翻 PPO； $r=\pi_{\text{new}}/\pi_{\text{old}}$ ，正负优势分情况讨论。
- terminated/truncated：翻 PPO 代码；截断不是自然终止，仍 bootstrap。
- continuous action：翻 DDPG/SAC；DDPG 确定性策略，SAC 最大熵随机策略。
- comparison：翻第 8 章；按模型/采样、target、on/off-policy、动作空间、优缺点对比算法。

11 常见激活函数总表

函数	公式	输出范围	常见用法	易错点
Sigmoid	$\sigma(x)=1/(1+\exp(-x))$	(0,1)	二分类概率；门控结构	大正/大负时梯度接近 0，深层隐藏层容易梯度消失。
Tanh	$\tanh(x)=(\exp(x)-\exp(-x))/(\exp(x)+\exp(-x))$	(-1,1)	RNN/连续动作范围；零中心输出	仍可能饱和；输出有正有负，比 sigmoid 更适合作隐藏状态。
ReLU	$\max(0,x)$	$[0,+\infty)$	MLP/CNN 隐藏层默认选择	$x<0$ 梯度为 0，可能出现死亡 ReLU。
Leaky ReLU	x if $x>0$ else αx	$(-\infty,+\infty)$	替代 ReLU，缓解死亡 ReLU	α 通常很小，如 0.01；不是输出概率。

ELU/SELU	$x \text{ if } x > 0 \text{ else } \alpha(\exp(x) - 1)$	$(-\alpha, +\infty)$	深层网络可选，改善负半轴输出	SELU 依赖特定初始化和网络条件。
GELU	$x * \Phi(x)$	约 $(-0.17, +\infty)$	Transformer/现代深层网络隐藏层	比 ReLU 更平滑，但手算题一般只问用途。
Softplus	$\log(1 + \exp(x))$	$(0, +\infty)$	需要正数输出时可用；ReLU 平滑版	输出不是概率，常用于方差/尺度参数。
Softmax	$\exp(z_i) / \sum_j \exp(z_j)$	$(0, 1)$, 和为 1	多分类；离散动作策略 $\pi(a s)$	要对一组 logits 一起算；不是逐元素 sigmoid。
Linear	$f(x) = x$	$(-\infty, +\infty)$	回归输出；V/Q 值输出	Q 值和价值不是概率，输出层不要乱加 sigmoid/softmax。

11.1 怎么选择：按网络位置判断

265. 看隐藏层：如果题目没有特殊说明，MLP/CNN 隐藏层优先写 ReLU；Transformer 类模型常见 GELU。
266. 看二分类输出层：如果输出是“属于某类的概率”，用 sigmoid，把 logit 压到 $(0, 1)$ 。
267. 看多分类或离散动作策略：如果输出是多个类别/动作的概率分布，用 softmax，使所有概率和为 1。
268. 看价值函数或 Q 网络输出：V(s)、Q(s,a) 可以是任意实数，通常输出层用 linear，不加 sigmoid/softmax。
269. 看连续动作 Actor：若环境动作范围是 $[-1, 1]$ ，最后常用 tanh；若动作范围是 $[low, high]$ ，再把 tanh 输出缩放至该范围。
270. 看是否需要正数参数：例如标准差、方差、尺度参数，可以用 softplus 或 exp 保证为正。
271. 看梯度问题：sigmoid/tanh 容易饱和，ReLU 计算简单但可能死亡，Leaky ReLU 给负半轴保留小梯度。

11.2 强化学习里怎么用

场景	推荐输出层	为什么
DQN/Q-learning 神经网络	linear	输出每个动作的 Q(s,a)，Q 值不是概率，可正可负，不应 softmax。
价值网络 Critic: V(s)	linear	状态价值是期望回报，范围由奖励决定，不固定在 0-1。
离散动作 Actor/PPO	softmax	输出 $\pi(a s)$ ，要形成动作概率分布并用于采样、log_prob、ratio。
连续动作 DDPG Actor	tanh + scale	Actor 输出确定性连续动作，tanh 先限制到 $[-1, 1]$ ，再映

		射到环境动作范围。
SAC 连续动作策略	tanh-squashed Gaussian	先采样高斯动作，再 tanh 压缩到动作边界，同时修正 log_prob。
隐藏层	ReLU/GELU	负责非线性特征提取；输出层再按任务决定。

11.3 例题：DQN 网络激活函数怎么写

题目：设计一个 DQN 网络，输入状态维度为 4，离散动作数为 2。说明隐藏层和输出层分别用什么激活函数，并解释原因。

- 272. 先判断任务：DQN 学的是动作价值函数 $Q(s,a)$ 。
- 273. 输入层只接收状态向量，不需要激活函数；它只是把状态送入网络。
- 274. 第一层线性层可写 `Linear(4,128)`，把 4 维状态映射到 128 维特征。
- 275. 隐藏层后接 ReLU： $h1 = \text{ReLU}(W1*s + b1)$ ，用于引入非线性。
- 276. 第二个隐藏层也可写 $h2 = \text{ReLU}(W2*h1 + b2)$ ，继续提取非线性特征。
- 277. 输出层写 `Linear(128,2)`，输出两个数： $[Q(s,a1), Q(s,a2)]$ 。
- 278. 输出层不加 softmax，因为 Q 值不是动作概率，而是期望回报，可能大于 1，也可能为负。
- 279. 输出层也不加 sigmoid，因为 sigmoid 会把 Q 值限制到 (0,1)，破坏价值大小。
- 280. 如果要选动作，再在网络输出的 Q 值上做 argmax，而不是用 softmax 概率。

DQN 输出层示例

```
class Qnet(nn.Module):
    def __init__(self, state_dim, hidden_dim, action_dim):
        super().__init__()
        self.fc1 = nn.Linear(state_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, action_dim)

    def forward(self, x):
        x = F.relu(self.fc1(x))
        return self.fc2(x) # linear Q values, no softmax
```

11.4 例题：PPO 离散策略为什么用 softmax

题目：离散动作空间有 4 个动作，Actor 网络输出 logits=[1,2,0,1]。说明为什么要用 softmax，并写出动作概率的计算方式。

- 281. PPO 的 Actor 需要给出策略 $\pi(a|s)$ ，也就是每个动作的概率。
- 282. 网络最后一层通常先输出 logits，logits 可以是任意实数，不是概率。
- 283. 要把 logits 变成概率分布，需要使用 softmax。
- 284. softmax 公式： $\pi(a_i|s) = \exp(z_i) / \sum_j \exp(z_j)$ 。
- 285. 代入本题：分母为 $\exp(1) + \exp(2) + \exp(0) + \exp(1)$ 。

286. 四个概率分别为 $\exp(1)/Z$, $\exp(2)/Z$, $\exp(0)/Z$, $\exp(1)/Z$ 。
287. 这样每个概率都大于 0, 并且四个概率相加等于 1。
288. PPO 更新时用 $\log_prob$ 和 old_log_prob 计算 $ratio = \exp(\log_prob - old_log_prob)$ 。
289. 所以离散 PPO 的输出层要能形成概率分布, softmax 正好满足这个要求。

11.5 例题：连续动作 Actor 为什么常用 tanh

题目：DDPG 环境动作范围是 $[-2, 2]$, Actor 网络最后输出 u 。如何把 u 变成合法动作？

290. 连续控制中, Actor 输出的是动作值, 不是离散动作概率。
291. 如果环境要求动作必须在 $[-2, 2]$, 直接输出 u 可能越界。
292. 先用 $\tanh(u)$ 把输出压到 $[-1, 1]$ 。
293. 再按动作范围缩放: $a = 2 * \tanh(u)$ 。
294. 若范围是 $[low, high]$, 通用缩放为 $a = low + (\tanh(u) + 1) * (high - low) / 2$ 。
295. $\tanh$ 的作用是保证动作合法, 避免网络输出超出环境动作边界。
296. 注意: 这里用 $\tanh$ 不是为了输出概率, 而是为了限制连续动作范围。

11.6 激活函数易错问答

- 为什么不能所有层都不用激活函数? 因为线性层叠加仍是线性变换, 无法表达复杂非线性关系。
- 为什么隐藏层常用 ReLU? 计算简单, 正半轴梯度稳定, 训练通常比 sigmoid/tanh 更快。
- 什么是死亡 ReLU? 某些神经元长期输入小于 0, 输出和梯度都为 0, 参数难以恢复更新。
- sigmoid 和 softmax 区别? sigmoid 常用于独立二分类/多标签; softmax 用于互斥多分类, 所有类别概率和为 1。
- Q 网络输出为什么不用 softmax? 因为 Q 值是价值, 不是概率; 选动作时对 Q 值取 $\argmax$ 。
- Actor 输出为什么有时用 softmax、有时用 tanh? 离散动作要概率分布用 softmax; 连续有界动作要范围限制用 tanh。